Pembahasan Singkat OSN 2022 Bidang Informatika

Daring, 7 Oktober 2022

Soal OSN 2022 Bidang Informatika

	Judul Soal	Pembuat Soal	Disiapkan oleh
1A	Membangun Pertahanan	Pikatan Arya Bramajati	R. Fausta Anugrah Dianparama
1B	Membangun Terowongan	Rama Aryasuta Pangestu	Abdul Malik Nurrokhman
1C	Membangun Mercusuar	Rama Aryasuta Pangestu	Rama Aryasuta Pangestu
2A	Membangun Jembatan	Mushthofa, Rama Aryasuta Pangestu	Mushthofa
2B	Membangun Gunungan Buah	Pikatan Arya Bramajati	Steven Novaryo
2C	Membangun Jalan	Pikatan Arya Bramajati	Pikatan Arya Bramajati

Statistik Soal

	Judul Soal	Pengumpulan	Rata-rata	Maks
1A	Membangun Pertahanan	674	49	100
1B	Membangun Terowongan	468	19	50
1C	Membangun Mercusuar	751	9	65
2A	Membangun Jembatan	955	20	100
2B	Membangun Gunungan Buah	694	17	64
2C	Membangun Jalan	725	23	66

Membangun Pertahanan

- Terdapat N lahan kosong dengan lahan ke-i memiliki panjang:
  - (i + 1) / 2 jika i ganjil.
  - N + 1 - (i / 2) jika i genap.
- Ada Q pertanyaan, dengan pertanyaan ke-j menanyakan banyaknya segmen lahan yang total panjangnya tepat X[j].

Membangun Pertahanan (1/4)

- Kita bisa lihat bahwa bentuk *array* dari panjang lahan-lahannya adalah:

  1, N, 2, N - 1, 3, N - 2, 4, N - 3, ...

- Kita bisa lihat bahwa:
  - Lahan 1 dan lahan 2 jumlahnya N + 1
  - Lahan 3 dan lahan 4 jumlahnya N + 1
  - ...
- Maka kita bisa memandang elemen-elemen pada array dengan mengelompokkannya menjadi pasangan-pasangan.
- Perhatikan juga bahwa setiap nilai dari 1 sampai N hanya muncul tepat sekali.

Membangun Pertahanan (2/4)

- Kita bisa konsiderasi setiap segmen (L, R) untuk setiap kasus berikut:
  - Jika L ganjil dan R genap, segmennya hanya berisi pasangan-pasangan berjumlah N + 1. Maka total panjangnya pasti kelipatan N + 1.
  - Jika L ganjil dan R ganjil, bentuknya seperti kasus pertama tetapi ditambahkan dengan satu elemen di kanannya. Elemen tambahannya adalah elemen yang ada di indeks ganjil, maka nilainya pasti tidak lebih dari ceil(N / 2). Total panjangnya pasti memiliki sisa tidak lebih dari ceil(N / 2) jika dibagi dengan N + 1.
  - Jika L genap dan R genap, bentuknya seperti kasus pertama tetapi ditambahkan dengan satu elemen di kirinya. Elemen tambahannya adalah elemen yang ada di indeks genap, maka nilainya pasti lebih dari ceil(N / 2). Total panjangnya pasti memiliki sisa lebih dari ceil(N / 2) jika dibagi dengan N + 1.

Membangun Pertahanan (3/4)

- Untuk L genap dan R ganjil, kita bisa melihat dengan sudut pandang lain.

1, N, 2, N - 1, 3, N - 2, 4, N - 3, ...

- Kita bisa lihat bahwa:
  - Lahan 2 dan lahan 3 jumlahnya N + 2
  - Lahan 4 dan lahan 5 jumlahnya N + 2
  - ...
- Maka dilihat dengan pengelompokan di atas, untuk L genap dan R ganjil, segmennya hanya berisi pasangan-pasangan berjumlah N + 2. Maka total panjangnya pasti kelipatan N + 2.

Membangun Pertahanan (4/4)

- Kita bisa menghitung jawaban untuk setiap pertanyaan dengan mengonsiderasi keempat kasus tersebut dan menghitung banyaknya segmen untuk masing-masing kasus dalam kompleksitas waktu O(1) menggunakan matematika sederhana.
- Kompleksitas waktu: O(Q)

Membangun Terowongan

- Soal interaktif.
- Terdapat N lokasi tambang.
- Antar lokasi bisa dihubungkan dengan terowongan dengan biaya A atau B.
- Boleh bertanya total biaya semua terowongan antar 2 set lokasi.
- Query (S, T), menghasilkan ans = sum($C_{u,v}$) for $u \in S$, $v \in T$.
- Buat terowongan untuk menghubungkan seluruh lokasi dengan biaya seminimum mungkin.
- MST? CC?

Subsoal 1 & 2

- N <= 5 (subsoal 1).
- N <= 60 (subsoal 2).
- $C_{u,v}$ = query ({u}, {v}).
- Cari semua nilai $C_{u,v}$.
- Hitung MST dengan algoritma prim / kruskal.
- Atau hitung CC dengan disjoint set union.
- Kompleksitas query: $O(N^2)$.

Subsoal 3

- A = 1, B = 2, untuk setiap lokasi u, paling banyak terdapat satu lokasi v (dengan u < v) yang memenuhi $C_{u,v}$ = A.
- Anggap lokasi v yang memenuhi adalah parent.
- Bentuk graf menjadi forest.
- Cari parent dari u dengan cara:
  - Dengan query ({u}, [L, R]), jika ans = B × (R - L + 1) maka parent tidak ada dalam range [L, R].
  - Cek apakah ada parent pada [u+1, N], jika tidak ada maka tidak punya parent.
  - Selain itu lakukan binary search pada [u+1, N] dengan cara tersebut.
- Untuk setiap 1 <= u < N:
  - Jika punya parent, maka buat terowongan (u, parent).
  - Selain itu, buat terowongan (u, N).
- Kompleksitas query: O(NlogN).

Subsoal 4

- A = 1, B = 2.
- Buat 2 buah set S = {1} dan T = [2, N].
- Untuk setiap u ∈ S lakukan binary search berikut:
  - Query ({u}, T), jika ans = B × |T| maka hapus u dari S.
  - Selain itu, lakukan binary search pada T dengan cara seperti subsoal 3.
  - Misalkan hasil dari binary search adalah v.
  - Buat terowongan (u, v).
  - Hapus v dari T, masukkan v ke dalam S.
- Jika S kosong dan T belum kosong:
  - Hapus salah satu u ∈ T dari T.
  - Buat terowongan (1, u).
  - Masukkan u ke dalam S.
  - Ulangi binary search di atas.
- Kompleksitas query: O(NlogN).

Subsoal 5

- Paling banyak terdapat 1 pasangan (u, v) (dengan u < v) yang memenuhi $C_{u,v}$ = A.
- Query ({1}, {2}), ({1}, {3}), dan ({2}, {3}).
- Jika hasil dari ketiganya ada yang berbeda, maka nilai yang berbeda adalah A, sisanya B.
- Selain itu, maka kita baru mendapatkan nilai B.
- Lakukan binary search berikut:
  - Tahap 1: pengecekan ke-i adalah query (S, T) dengan u ∈ S jika (u>>i)&1, selain itu u ∈ T. Jika ans = B × |S| × |T| maka lanjut ke pengecekan berikutnya. Selain itu, lanjut ke tahap berikutnya. Apabila tidak menemukan A dalam log(N) pengecekan berarti tidak ada nilai A.
  - Tahap 2: binary search set S sampai tersisa 1 node. Misalkan itu u.
- Buat terowongan (i, u) untuk i ≠ u.
- Kompleksitas query: O(2logN).

Subsoal 6 (parsial)

- Mendapatkan > 85 poin.
- Menggunakan cara subsoal 5 untuk mencari nilai A dan B dengan rincian:
  - Di awal hanya perlu query ({1}, {2}). Anggap B = ans, lalu lakukan tahap 1 dan 2.
  - Dilanjutkan tahap 3: binary search set T sampai tersisa 1 node. Misalkan itu v.
  - Query ({u}, {v}) untuk mendapatkan nilai A = ans.
  - Jika ternyata A > B maka tukar nilai A dan B.
  - Diselesaikan dalam O(3logN) query.
- Menggunakan cara subsoal 4 untuk melakukan algoritma prim dalam O(NlogN) query.
- Kompleksitas query: O((N+3)logN).

Subsoal 6 (penuh)

- Mencari nilai A dan B bisa dilakukan sembari melakukan algoritma prim.
- Query ({1}, {2}) untuk mendapatkan salah satu nilai antara A atau B, misalkan itu Z.
- Query pertama yang dilakukan ketika melakukan algoritma prim adalah query ({1}, [2, N-1]}.
- Bagi menjadi 3 kasus:
  - Jika ans < Z × (N - 1), maka B = Z. Lanjutkan binary search yang ada pada algoritma prim. Setelah mendapatkan v pada binary search, query ({1}, {v}) lalu didapatkan A = ans.
  - Jika ans > Z × (N - 1), maka A = Z. Query ({1}, {u}} untuk u > 1, jika ans = A maka pindahkan u dari T ke S. Selain itu, kita mengetahui nilai B = ans dan melanjutkan algoritma prim.
  - Jika ans = Z × (N - 1), maka hapus 1 dari S dan pindahkan 2 dari T ke S. Ulangi query ({u}, [u+1, N]) untuk u > 1 sampai ans ≠ Z × (N - u).
    - Jika ans > Z × (N - u), maka buat terowongan (1, u) untuk u > 1.
    - Jika ans < Z × (N - u), maka lakukan cara seperti pada kasus 1.
- Kompleksitas query: O(NlogN).

Membangun Mercusuar

- Terdapat N mercusuar dengan tinggi A[1], A[2], ..., A[N].
- Dapat meningkatkan tinggi mercusuar dengan harga 1 per meter.
- Ingin agar untuk setiap mercusuar i, terdapat setidaknya satu mercusuar j sehingga A[i] = A[j] dan setiap mercusuar min(i, j) < k < max(i, j) memiliki tinggi lebih rendah.

Membangun Mercusuar

- Terdapat N mercusuar dengan tinggi A[1], A[2], ..., A[N].
- Dapat meningkatkan tinggi mercusuar dengan harga 1 per meter.
- Ingin agar untuk setiap mercusuar i, terdapat setidaknya satu mercusuar j sehingga A[i] = A[j] dan setiap mercusuar min(i, j) < k < max(i, j) memiliki tinggi lebih rendah.
- Perhatikan bahwa kita dapat membuat semua ketinggian saling berbeda dengan A[i] := A[i] + iε, di mana ε adalah konstanta yang sangat kecil.
- Jadi, dalam pembahasan selanjutnya, kita selalu dapat mengasumsikan semua ketinggian A[i] berbeda.

Subsoal 2

- $N \leq 8$
- Kita dapat mencoba mengubah ketinggian setiap mercusuar ke semua N ketinggian yang mungkin.
- Ada N! cara untuk melakukan ini, karena kita hanya dapat meningkatkan ketinggian setiap mercusuar.
- Untuk setiap cara meninggikan mercusuar, kita dapat memeriksa apakah konfigurasi tersebut valid dalam O(N^3), O(N^2), atau O(N).

Subsoal 6

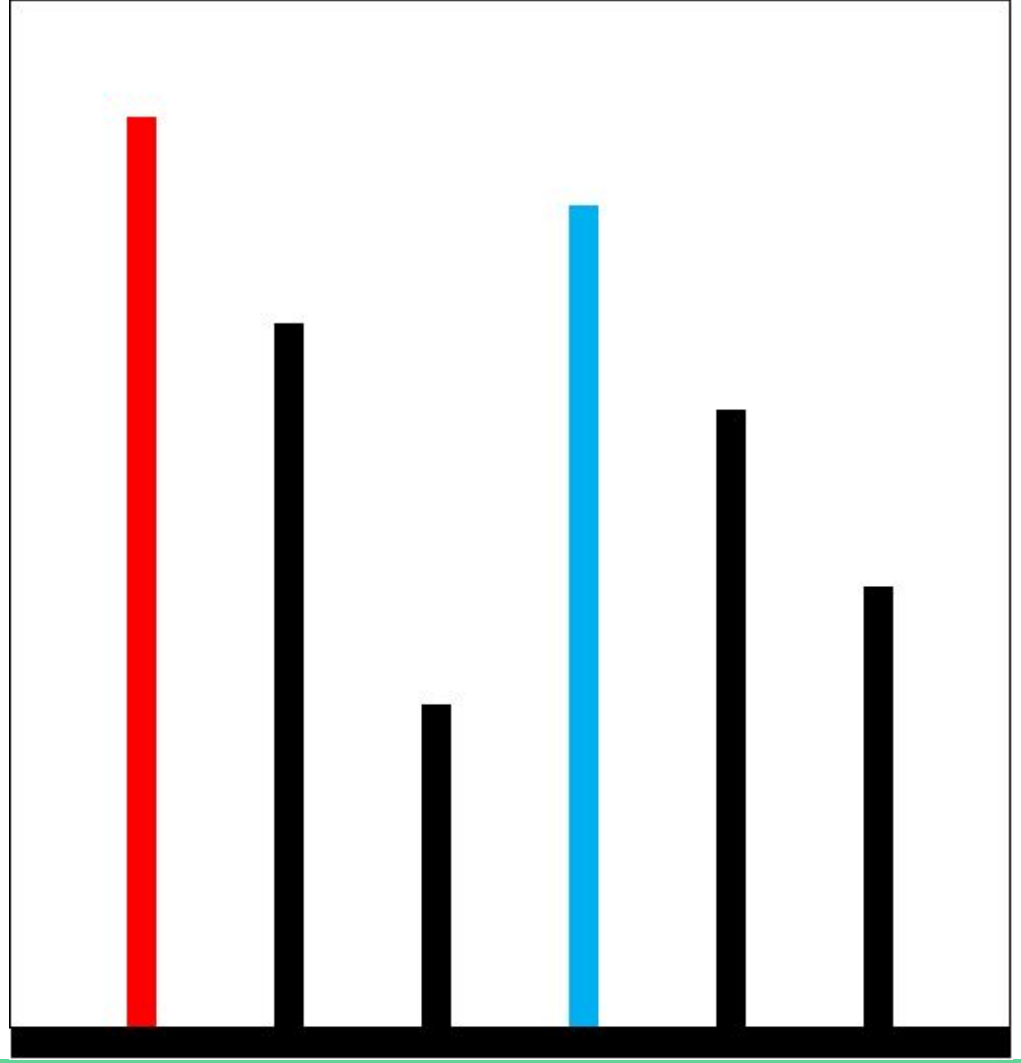
- $N \leq 80$
- Mari kita pertimbangkan algoritme berikut untuk memeriksa apakah sebuah konfigurasi mercusuar valid.
- Misalkan kita memeriksa apakah subarray [l,r] valid. Awalnya, ini [1, N].
- Misalkan m = max(A[l], A[l+1], .., A[r]).
- Misalkan s[1] < s[2] < ...< s[k] adalah indeks di mana A[s[i]] = m.
- Jika k = 1, seluruh konfigurasi tidak valid. Jika k > 1, secara rekursif kita periksa subarray [l, s[1] - 1], [s[1] + 1, s[2] - 1], ..., [s[k] + 1, r]. Setiap subarray independen karena s[i] adalah elemen maksimum.

Subsoal 6

- Definisikan dp(l, r, x) sebagai biaya minimum untuk membuat subarray [l,r] valid, jika kita juga dapat meningkatkan ketinggian mercusuar menjadi x secara langsung untuk membuatnya valid (dengan kata lain, kita memiliki mercusuar dengan tinggi x di luar subarray).
- Misal m adalah indeks dengan A[m] maksimum dengan $l \leq m \leq r$.
- Jika ketinggian mercusuar m diubah, maka kita pasti meningkatkan ketinggian menjadi x. Lalu, kita melakukan rekursi ke dp(l, m - 1, x) dan dp(m + 1, r, x).
- Jika ketinggian mercusuar m tidak berubah, maka kita harus memilih indeks lain m' lain, kemudian menambah tinggi mercusuar m' menjadi A[m]. Kemudian, rekursi ke dp(l, m - 1, A[m]), dp(m + 1, m' - 1, A[m]), dan dp(m' + 1, r, A[m]).

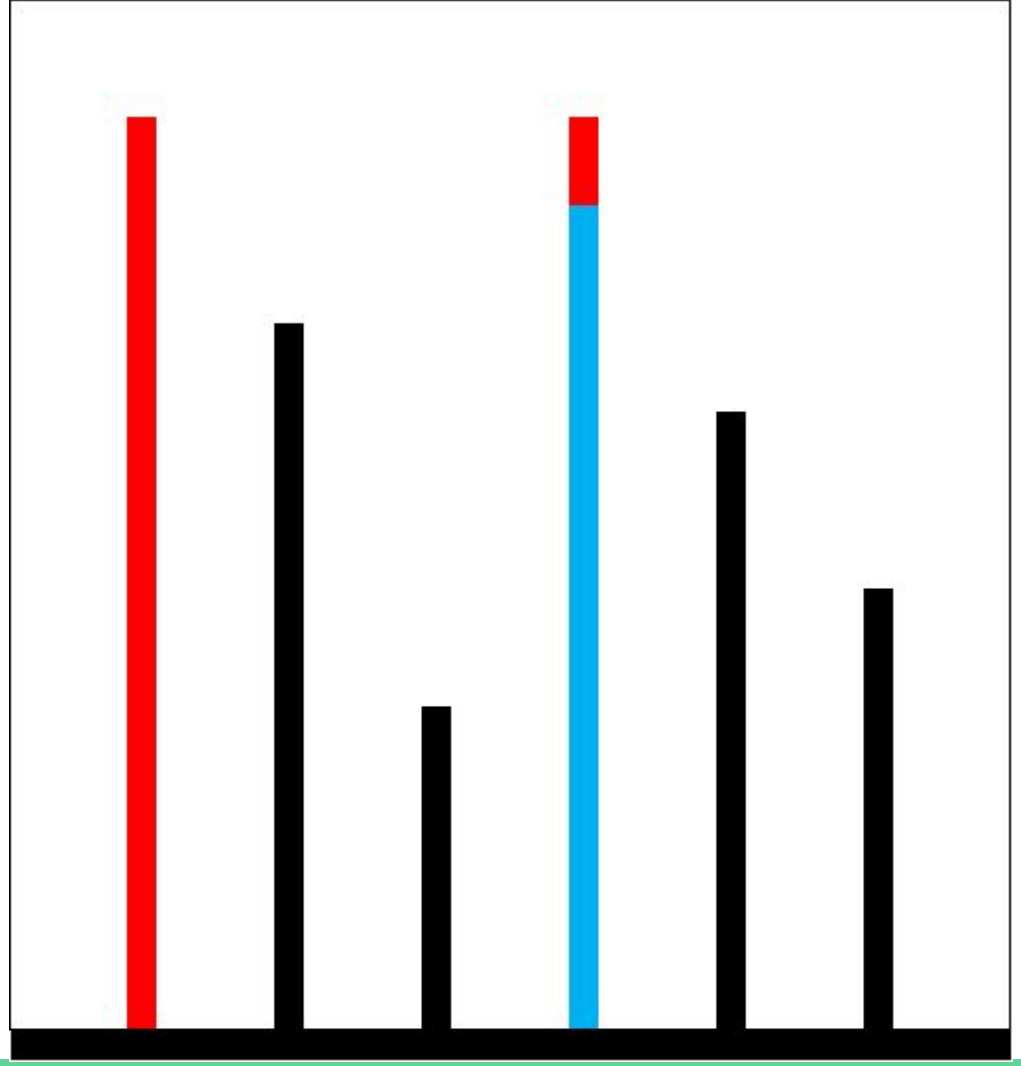
State saat ini

- **x = merah**
- **m = biru**



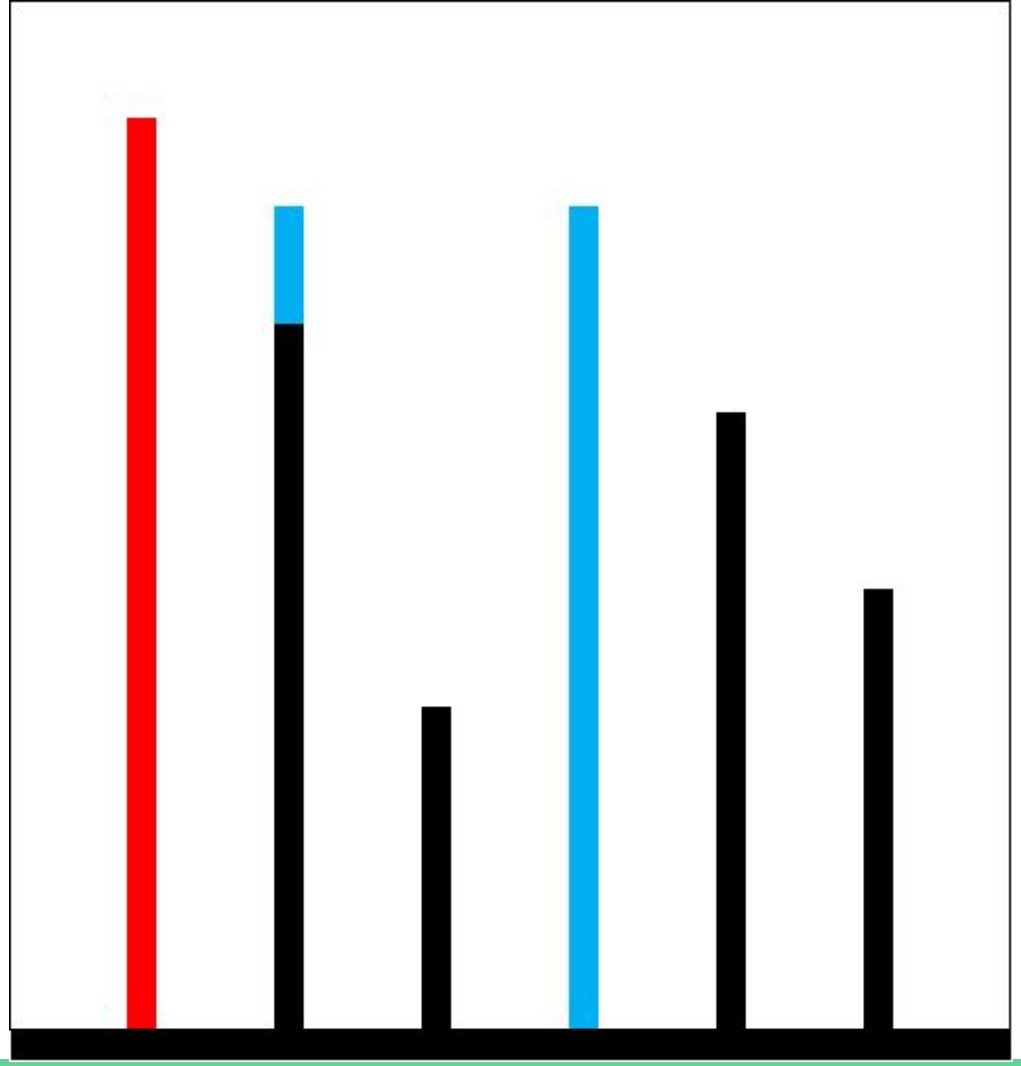
Transisi 1

- **x = merah**
- **m = biru**



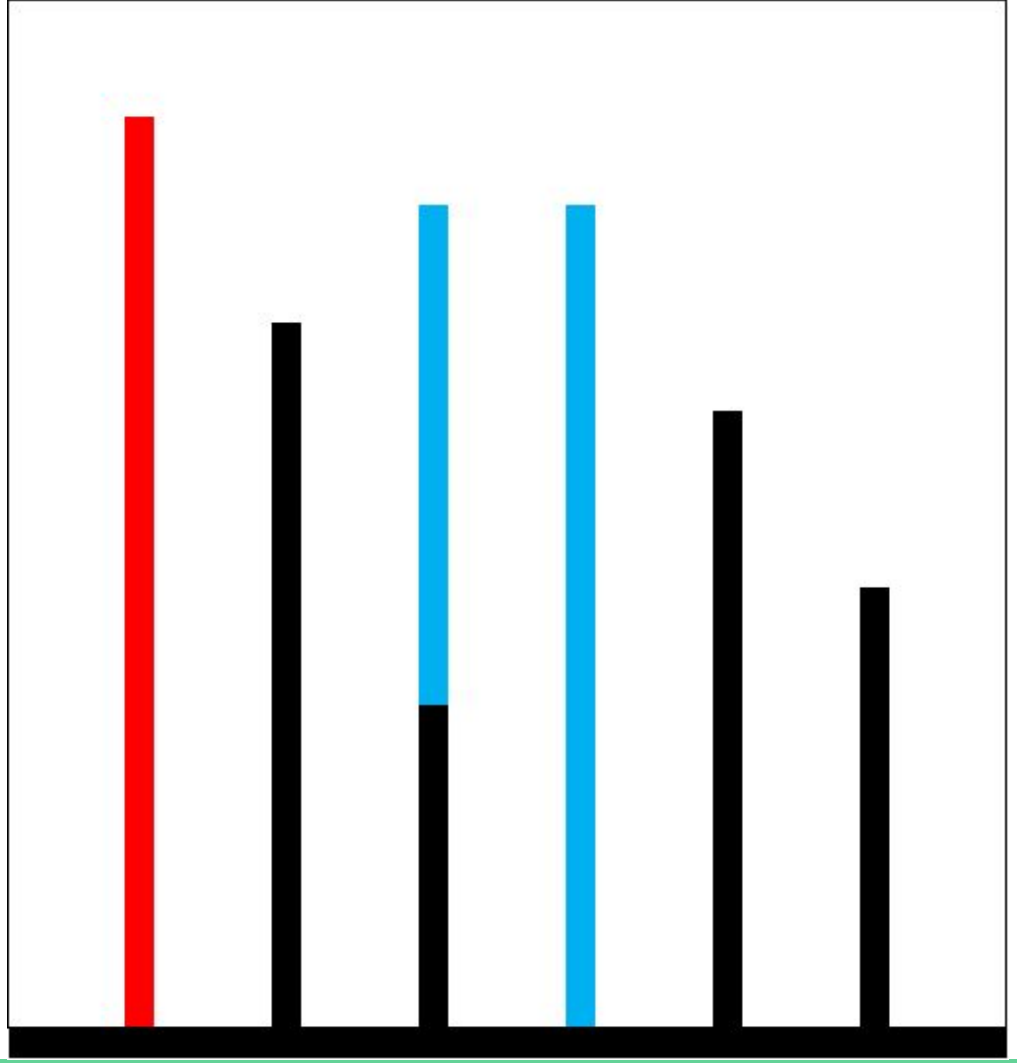
Transisi 2

- **x = merah**
- **m = biru**



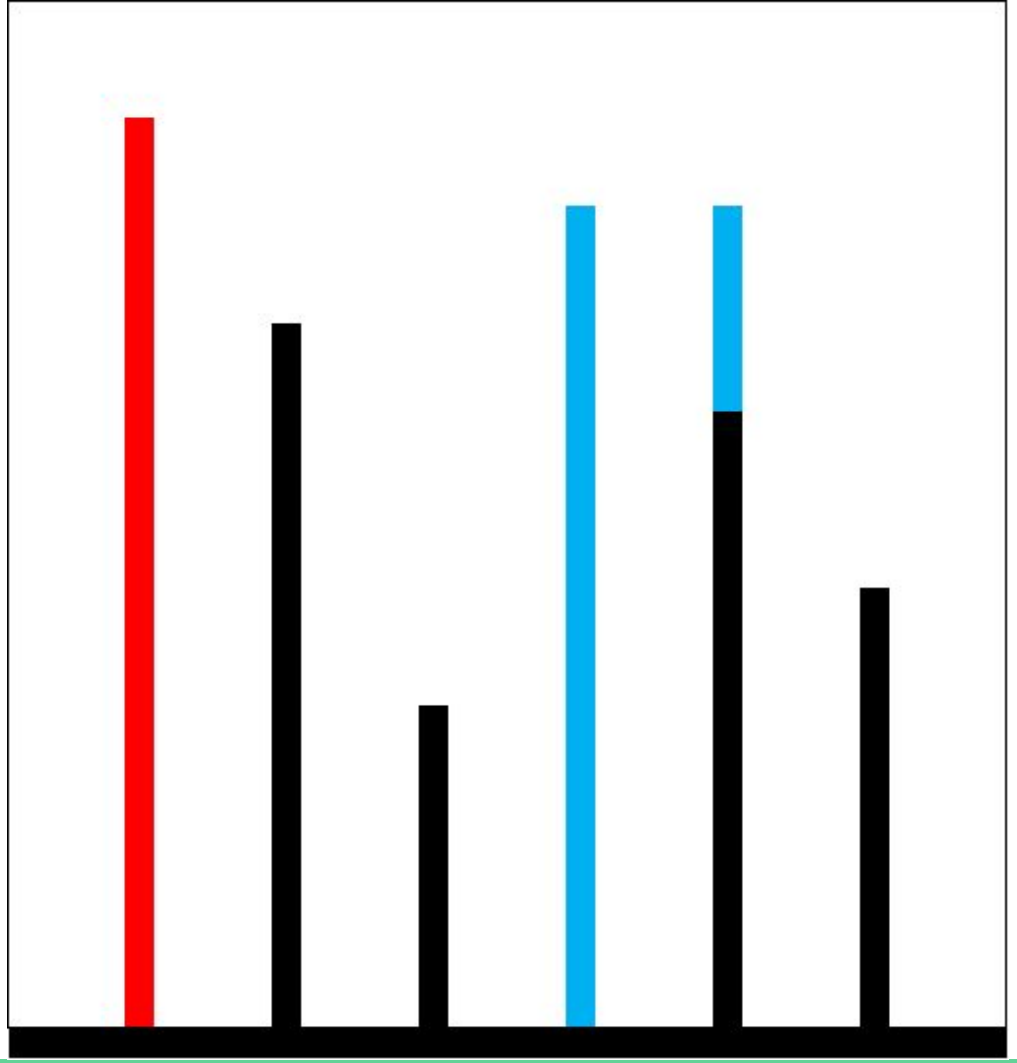
Transisi 2

- **x = merah**
- **m = biru**



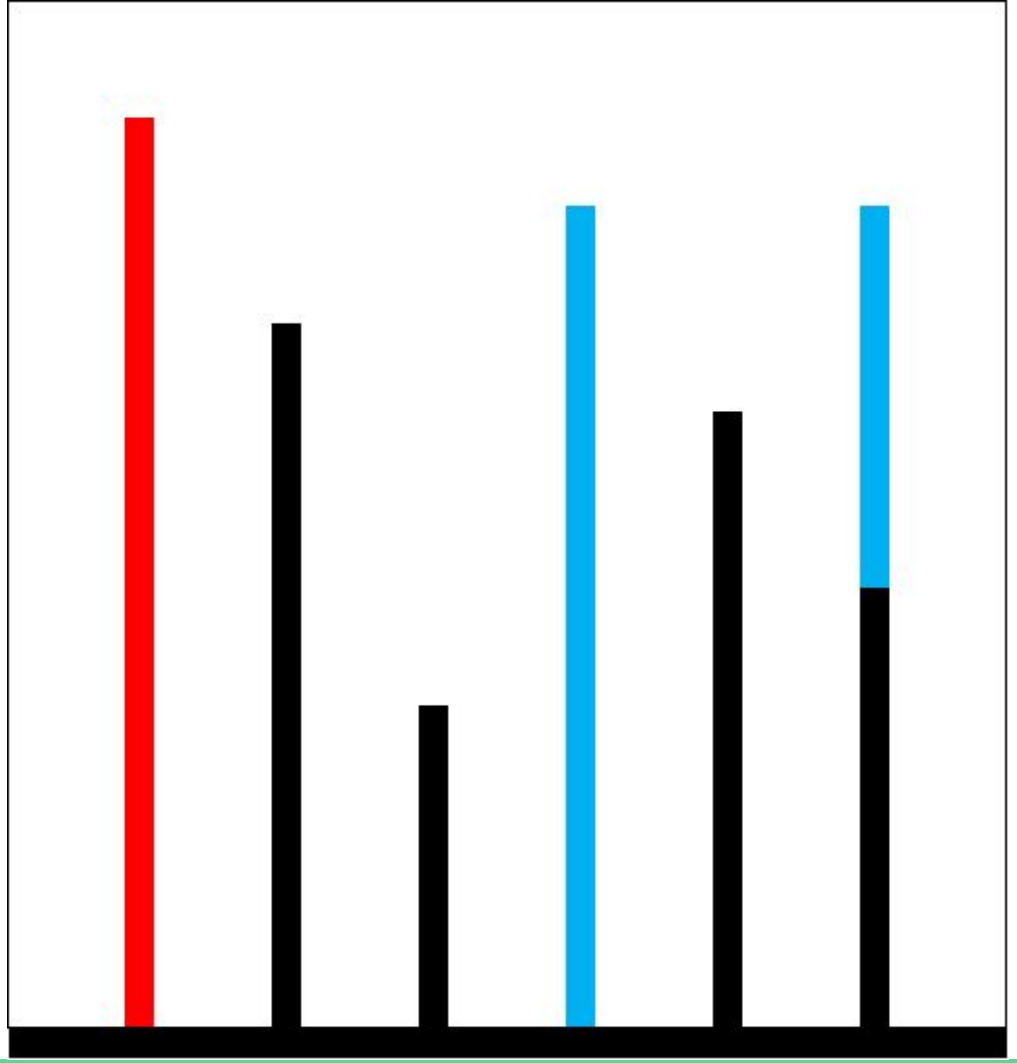
Transisi 2

- **x = merah**
- **m = biru**



Transisi 2

- **x = merah**
- **m = biru**



Subsoal 6

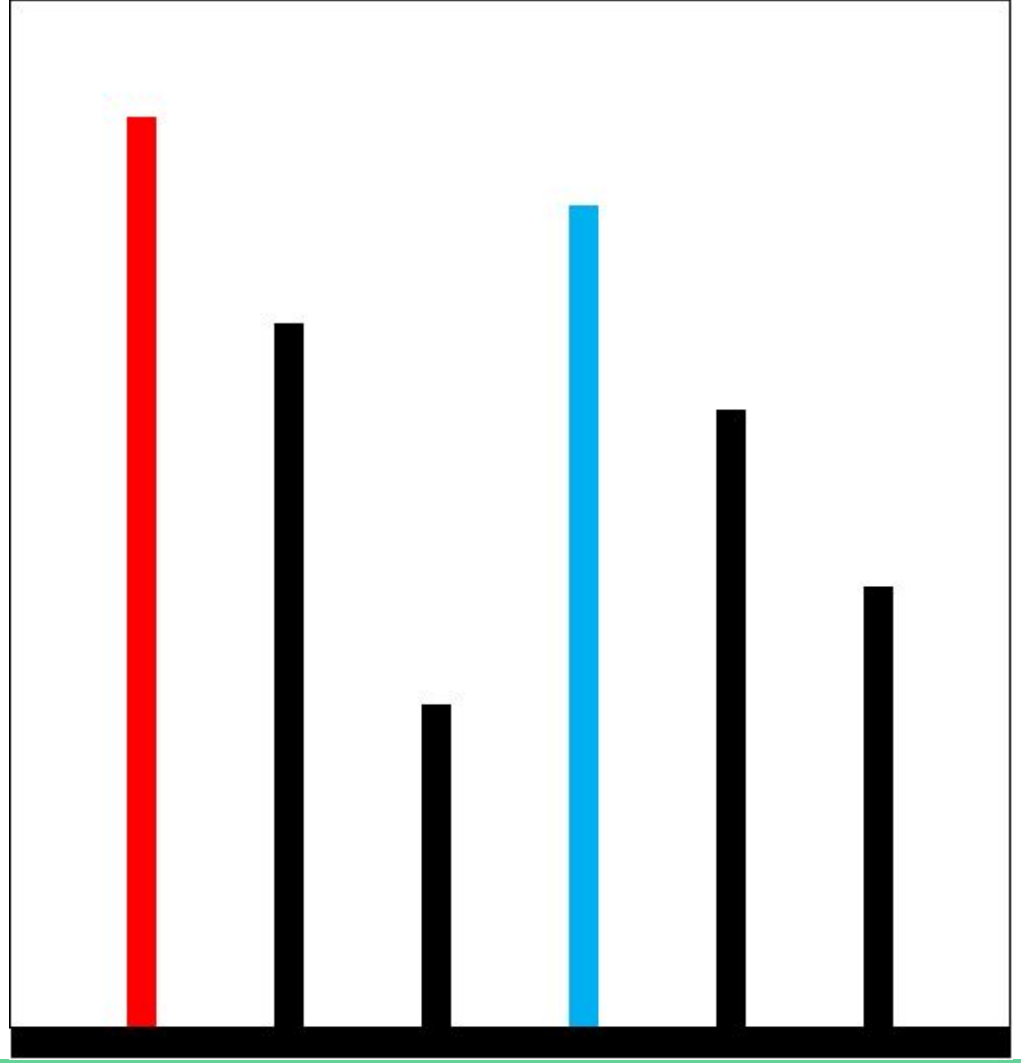
- Ada O(N^3) state pada DP
- Setiap transisi O(N)
- Kompleksitas waktu O(N^4)

Observasi

- Misal mercusuar l-1 tidak berubah tinggi, dan kita ingin pada subarray [l, r], terdapat satu mercusuar yang ditingkatkan tingginya menjadi A[l-1]
- Selalu terdapat konfigurasi optimal di mana mercusuar x yang ditingkatkan tingginya menjad A[l-1] memenuhi $x \leq m$, jika mercusuar l-1 tidak valid

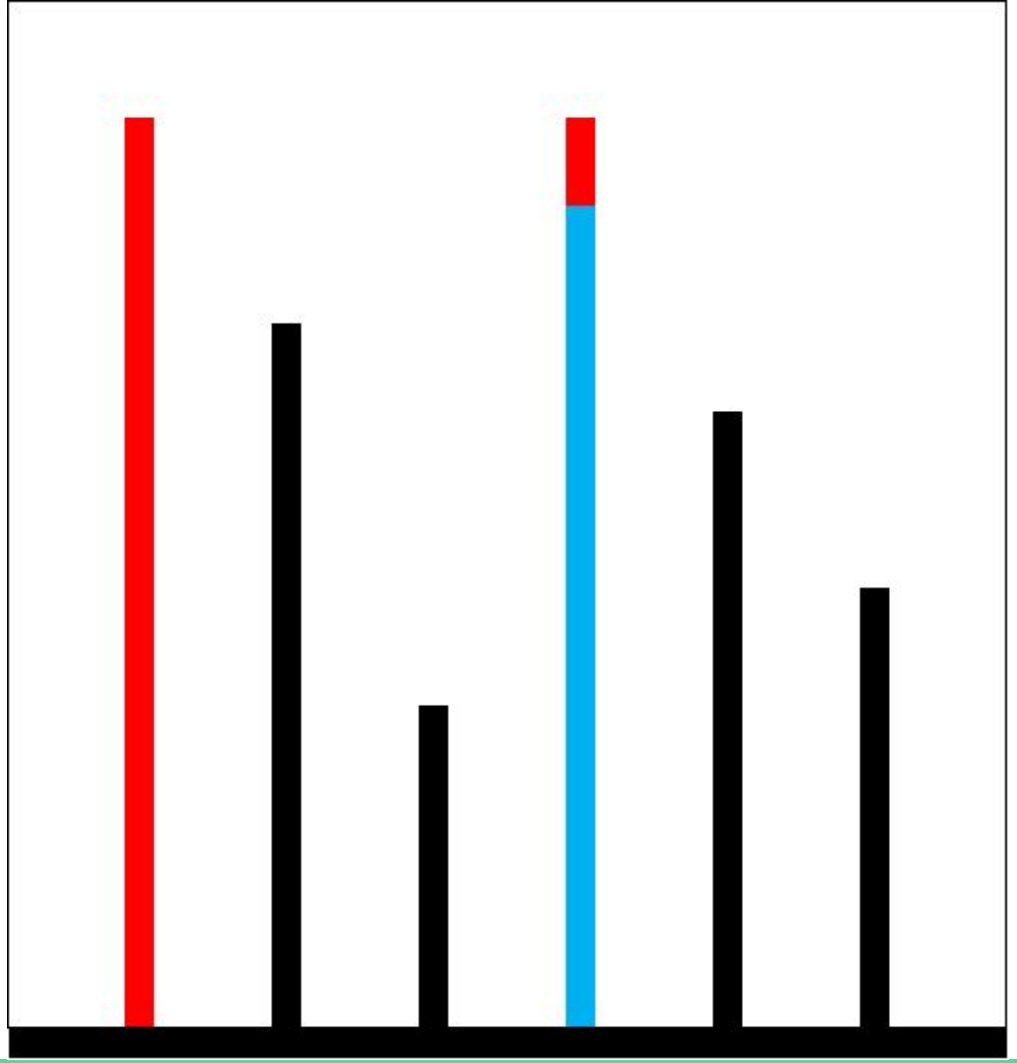
State saat ini

- **l-1 = merah**
- **m = biru**



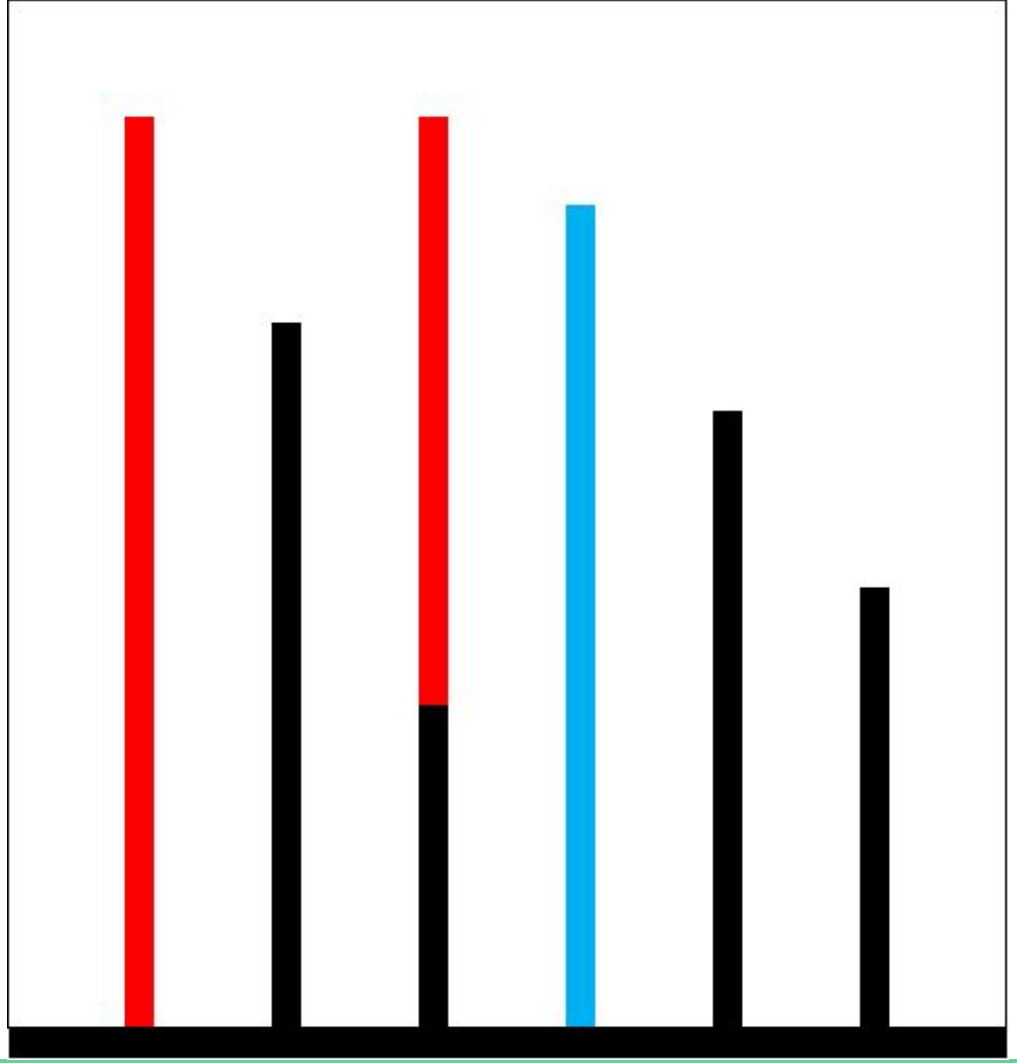
Kasus 1: x = m

- **I-1 = merah**
- **m = biru**



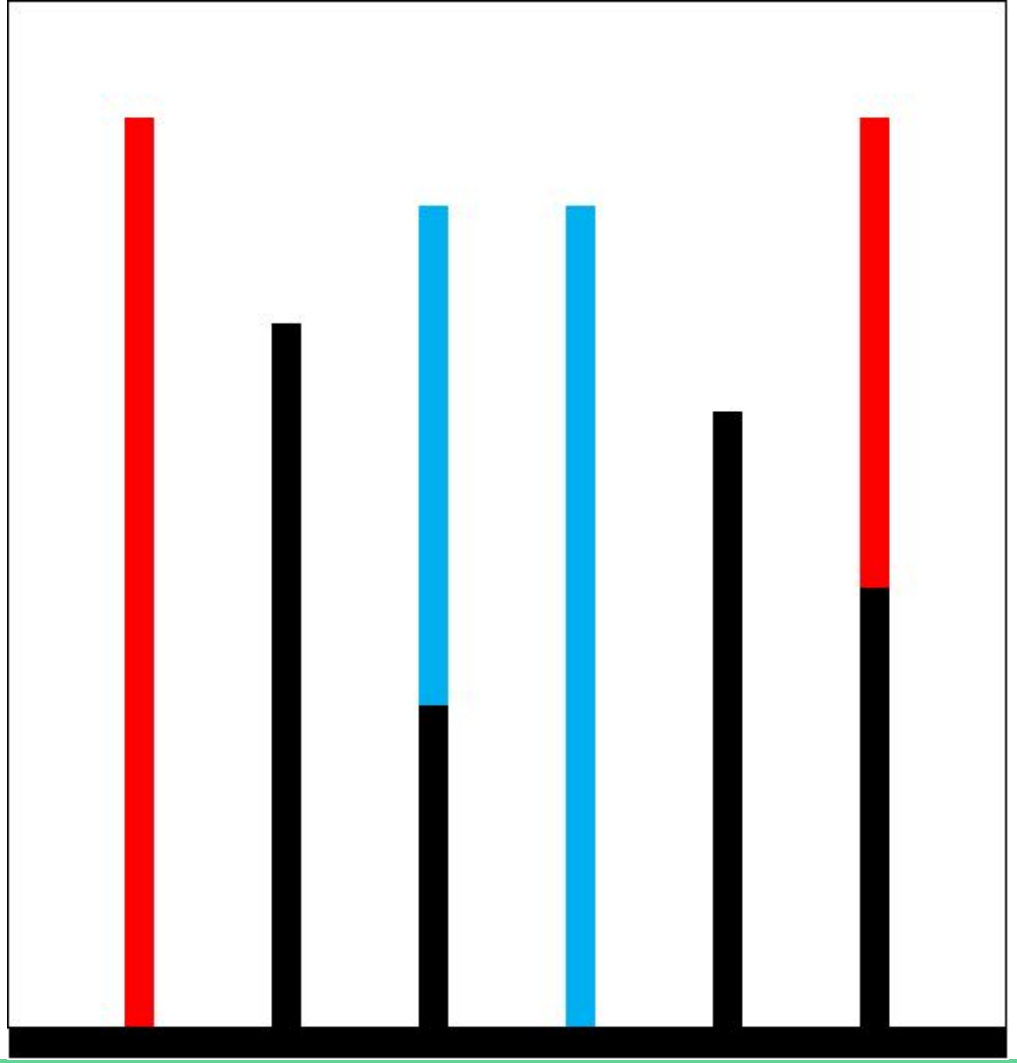
Kasus 2: x < m

- **I-1 = merah**
- **m = biru**



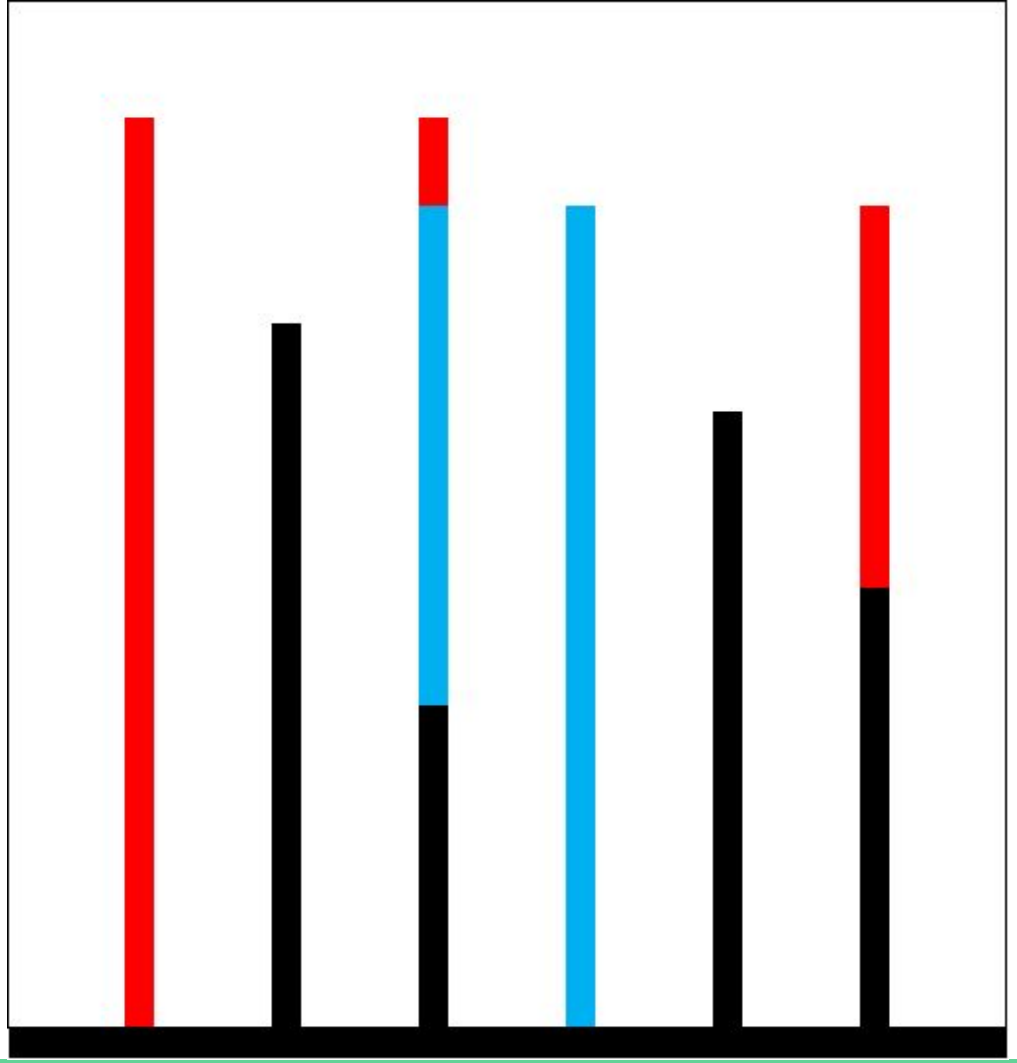
Kasus 3: x > m

- **I-1 = merah**
- **m = biru**



Kasus 3: $x > m \rightarrow x \leq m$

- **I-1 = merah**
- **m = biru**



Observasi

- Misal mercusuar l-1 tidak berubah tinggi, dan kita ingin pada subarray [l, r], terdapat satu mercusuar yang ditingkatkan tingginya menjadi A[l-1]
- Selalu terdapat konfigurasi optimal di mana mercusuar x yang ditingkatkan tingginya menjad A[l-1] memenuhi $x \leq m$, jika mercusuar l-1 tidak valid
- Selalu terdapat konfigurasi optimal di mana mercusuar x yang ditingkatkan tingginya menjad A[r+1] memenuhi $x \geq m$, jika mercusuar r+1 tidak valid

Subsoal 8

- $N \leq 300$
- Definisikan dp(l, r, x, bl, br) di mana:
- Jika bl = br = 0, maka sama dengan dp(l, r, x).
- Jika bl = 1, maka di subarray [l, r], kita harus menaikkan setidaknya satu indeks ke ketinggian A[l-1], karena saat ini mercusuar l-1 tidak valid.
- Jika br = 1, maka di subarray [l, r], kita harus menaikkan setidaknya satu indeks ke ketinggian A[r+1], karena saat ini mercusuar r+1 tidak valid.

Subsoal 8

- Transisi: memilih untuk tidak menambah ketinggian mercusuar m. Oleh karena itu, salah satu dari bagian kiri atau kanan yang dipisahkan oleh mercusuar m harus memiliki mercusuar dengan ketinggian akhir A[m].
- Transisi: kita meningkatkan ketinggian mercusuar m ke ketinggian x.
- Transisi: kita meningkatkan ketinggian mercusuar m ke ketinggian A[l-1], jika mercusuar l-1 tidak valid.
- Transisi: kita meningkatkan ketinggian mercusuar m ke ketinggian A[r+1], jika mercusuar r+1 tidak valid.

Subsoal 8

- Kita dapat mengimplementasi transisi di atas sehingga kita selalu rekursi ke dp(l, m-1, x', bl', br') dan dp(m+1, r, x', bl', br')
- Hanya terdapat O(N) pasangan (l, r) pada DP

Subsoal 8

- Kita dapat mengimplementasi transisi di atas sehingga kita selalu rekursi ke dp(l, m-1, x', bl', br') dan dp(m+1, r, x', bl', br')
- Hanya terdapat O(N) pasangan (l, r) pada DP

- Jumlah states O(N^2), dan transisi mencari nilai m dalam O(N)
- Dapat menyimpan state (l, r) dalam `std::map`
- Kompleksitas waktu O(N^3)

Subsoal 9

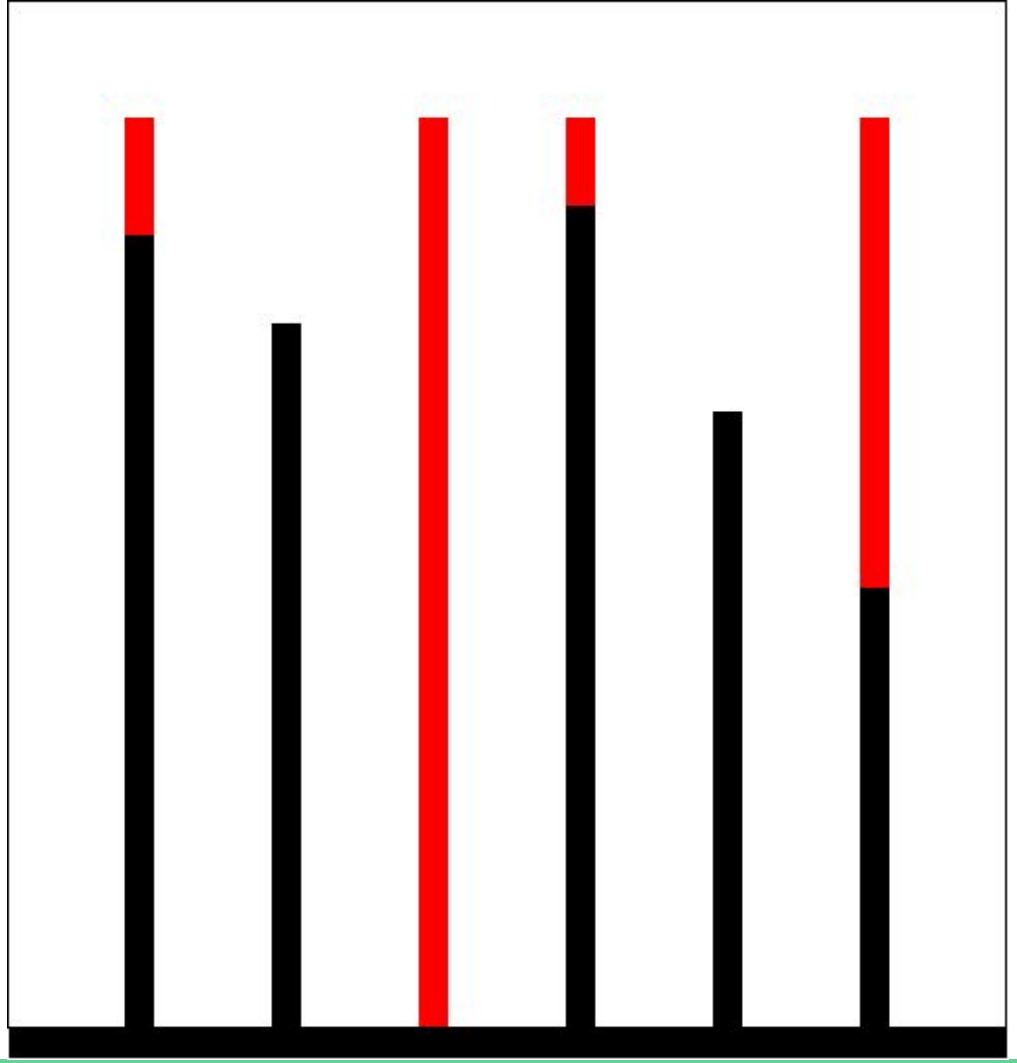
- $N \leq 1500$
- Misalkan kita memiliki fungsi `solve(l, r)` yang mengembalikan nilai dp(x, bl, br) pada subarray berupa array tiga dimensi.
- Kita dapat menemukan m dalam waktu O(N) dengan mengecek semua indeks.
- Kita memanggil `lft = solve(l, m-1)` dan `rgt = solve(m+1, r)` secara rekursif.
- Perhatikan bahwa semua transisi dari dp(l, r, x, bl, br) hanya memerlukan nilai dari `lft` atau `rgt`.
- Maka, kita tidak perlu lagi menyimpan DP dalam `std::map`.
- Kompleksitas waktu: O(N^2).

Observasi

- Misal m adalah mercusuar tertinggi pada awalnya, dan tinggi awal semua berbeda.
- Nilai maksimum pada konfigurasi akhir pasti A[m].
- Misal c adalah banyak kemunculan nilai A[m] pada konfigurasi akhir.
- Jika $c \geq 4$, bisa memilih sebuah kelompok di prefix atau suffix ukuran ≥ 2 yang tidak mengandung m, kemudian mengurangi tinggi pada konfigurasi akhir.

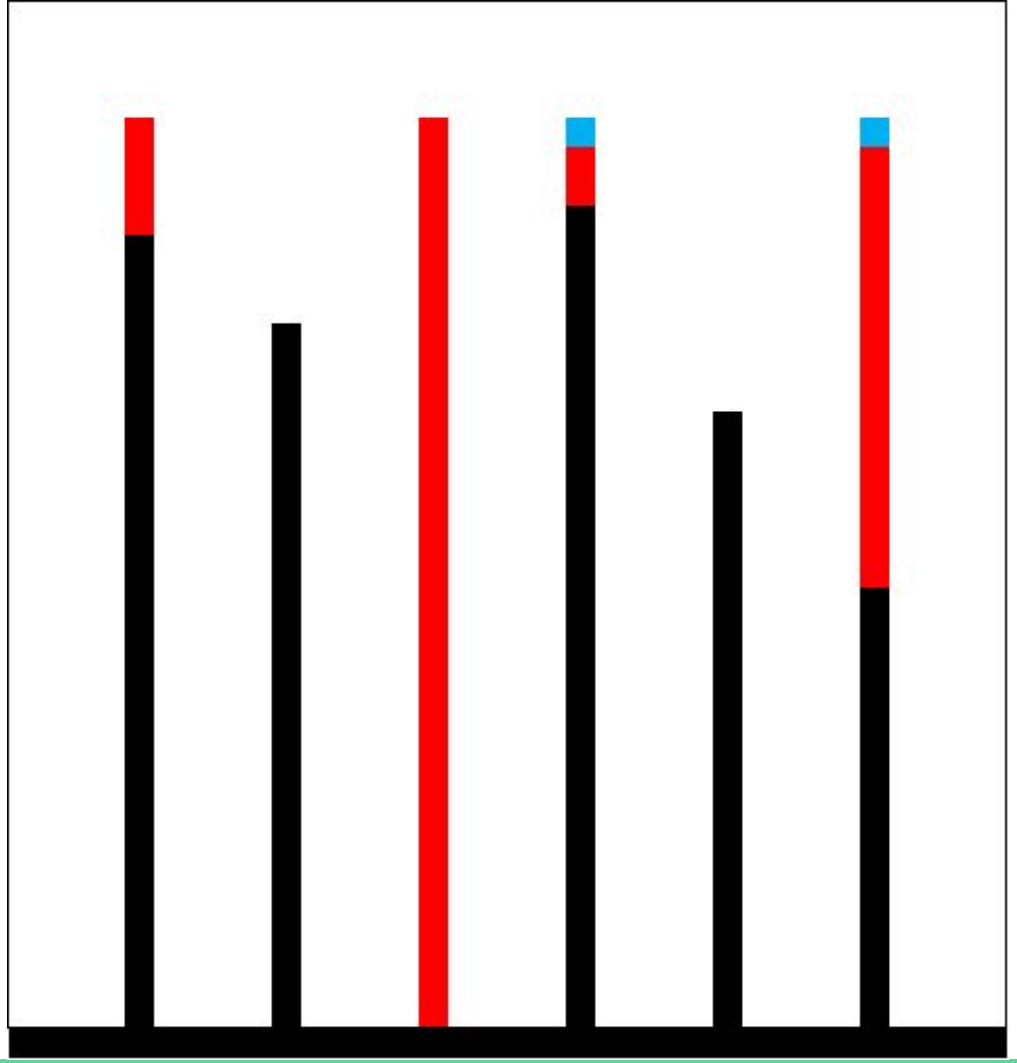
Jika $c \geq 4$

- **m = merah**



Total harga berkurang

- **m = merah**
- **hapus = biru**



Observasi

- Misal m adalah mercusuar tertinggi pada awalnya, dan tinggi awal semua berbeda.
- Nilai maksimum pada konfigurasi akhir pasti A[m].
- Misal c adalah banyak kemunculan nilai A[m] pada konfigurasi akhir.
- Jika $c \geq 4$, bisa memilih sebuah kelompok di prefix atau suffix ukuran ≥ 2 yang tidak mengandung m, kemudian mengurangi tinggi pada konfigurasi akhir.
- Maka $c \geq 4$ selalu tidak optimal. Pasti $c \leq 3$.

Solusi Penuh

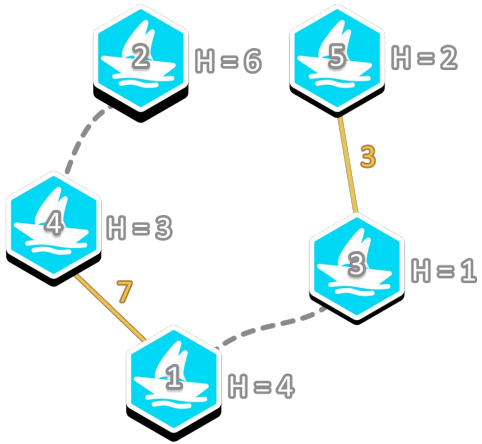
- $N \leq 100\,000$
- Kita menyimpan x sebagai state karena digunakan untuk menghitung x - A[m] dalam transisi.
- Hitung secara terpisah: ketika kita menginisialisasi elemen menjadi parameter x dalam DP, kita menambahkan kontribusi +x.
- Ketika kita menambah ketinggian mercusuar m, kontribusi -A[m].
- Dengan observasi $c \leq 3$, kita hanya perlu menyimpan apakah kita masih boleh meningkatkan 0, 1, atau 2 ketinggian mercusuar ke suatu nilai x (karena salah satu kemunculan pasti dari mercusuar yang ketinggian pada awalnya sama).

Solusi Penuh

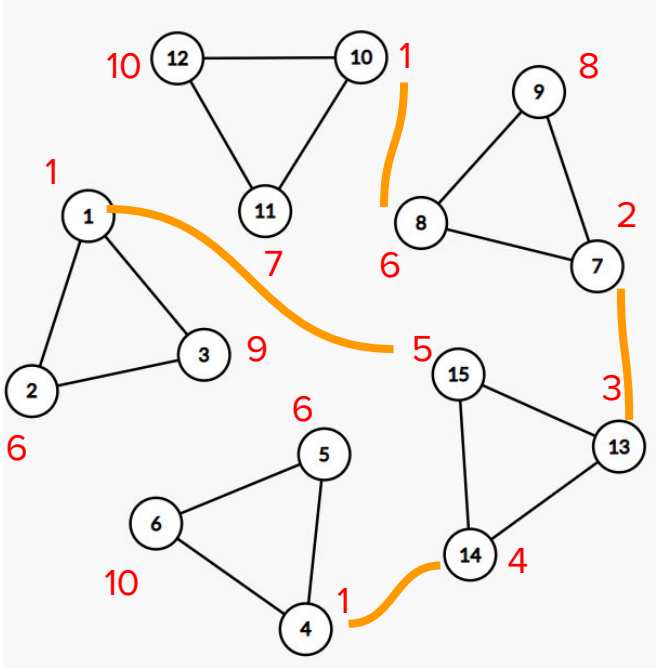
- Definisikan dp(l, r, c, bl, br), di mana kita dapat membuat c mercusuar valid dengan menambahkan kontribusi -A[m].
- Transisi serupa dengan DP O(N^2)
- Jumlah states hanya N * 3 * 2 * 2 = 12N
- Setiap transisi dapat dilakukan dalam O(1)
- Untuk mencari m, bisa menggunakan Segment Tree
- Kompleksitas waktu O(N log N)

Membangun Jembatan

- Diberikan *N* buah pelabuhan, dan *M* buah jalan yang masing-masing menghubungkan 2 buah pelabuhan. Setiap pelabuhan *i* memiliki *cost* H_i
- Bangun beberapa *jembatan* sedemikian rupa sehingga
  - Semua pelabuhan terhubung (lewat jalan atau jembatan)
  - Tidak ada dua pelabuhan yang terhubung dengan 2 atau lebih jembatan
  - Total biaya pembangunan seminimal mungkin. Biaya membangun jembatan antar pelabuhan *i* dan pelabuhan *j* adalah $H_i + H_j$
- Jika Q = 1, juga tuliskan daftar jembatan yang diperlukan



Subsoal 1



Solusi optimal (salah satu kemungkinan):

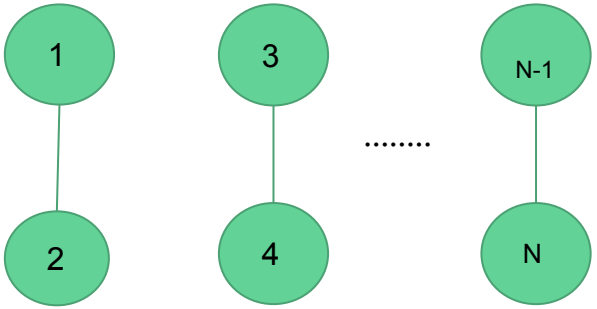
- H[7] + H[13] = 2 + 3 = 5
- H[10] + H[8] = 1 + 6 = 7
- H[1] + H[15] = 1 + 5 = 6
- H[4] + H[14] = 1 + 4 = 5
- Total = 5 + 7 + 6 + 5 = 23
- Jembatan:
  - 7 13
  - 10 8
  - 1 15
  - 4 14

Subsoal 2

- Pelabuhan 1 s/d N - 1 sudah terhubung, tinggal menghubungkan pelabuhan N
  - Cukup membangun 1 buah jembatan dari pelabuhan i, 1 <= i <= N-1, ke pelabuhan N
  - Biaya yang diperlukan = H[N] + H[i]
- Biaya terkecil didapat dengan mencari nilai H[i] terkecil untuk 1 <= i <= N-1, misal didapatkan dari $H[i_{min}]$
- Hubungkan pelabuhan i_{min} dengan N

Subsoal 3

- N genap, dan graf pelabuhan berbentuk sebagai berikut



- Terdapat N/2 “komponen” terpisah.
- **Observasi**: semua pelabuhan, kecuali 2 di antaranya, harus dihubungkan
- *Greedy*: Pilih 2 terbesar yang tidak dihubungkan, sisanya harus digunakan

Subsoal 4

- Bentuk graf hubungan antar pelabuhan melalui jalan adalah bebas, namun biaya di setiap pelabuhan selalu H = 1
- **Observasi**: Tidak perlu melakukan pemilihan / optimasi untuk mencari pelabuhan dengan biaya terkecil, yang penting semua saling terhubung
- Dapatkan dulu semua *connected component* dari graf input (dengan algoritme yang efisien [di bawah $O(N^2)$], misal DFS atau *union-find-disjoint-set*)
- **Observasi**: Misalkan ada C buah komponen, maka kita perlu membangun C - 1 jembatan (membuat *spanning tree* yang menghubungkan semua komponen), sehingga total biaya selalu 2(C-1), karena ada 2(C-1) pelabuhan berbeda yang harus dihubungkan
    - Jika 2(C-1) > N, konstruksi tidak dapat dilakukan
- Konstruksi: ambil minimal satu pelabuhan dari setiap komponen, sisanya bebas diambil, asal total = 2(C-1)

Subsoal 5 (+ subsoal 2)

- Dapatkan semua komponen dalam graf (boleh $O(N^2)$)
- Karena hanya ada ***maksimal*** 2 komponen, maka kita hanya perlu membangun **maksimal** 1 jembatan
- Jika jumlah komponen = 1, total biaya = 0 dan selesai
- Selainnya: untuk mendapatkan nilai total minimum, cukup dicari nilai H terkecil di masing-masing komponen lalu dijumlahkan
- Konstruksi 1 buah jembatan dengan memilih dua indeks i dan j dimana H_i adalah nilai terkecil di komponen pertama dan H_j adalah nilai terkecil di komponen kedua: jembatan = i - j

Observasi utama (Subsoal 6, 7 dan solusi penuh)

- Untuk menghubungkan semua pelabuhan, kita harus menghubungkan semua *connected component* dari graf input. Misalkan ada C komponen:
  - Jika C = 1, total biaya = 0, tidak perlu membangun jembatan
- Perlu membuat tree dari komponen-komponen tersebut ➜ C - 1 jembatan. Berarti total ada 2(C-1) = 2C-2 pelabuhan (karena tidak boleh dipakai 2x) yang harus dihubungkan
  - Jika 2C-2 > N, tidak mungkin menghubungkan semua pelabuhan (output “-1”)
- **Lemma**: Misalkan dari setiap komponen i kita memilih J[i] pelabuhan, maka semua pelabuhan akan terhubung sesuai kondisi soal jika dan hanya jika:
  - $J[i] \geq 1, 1 \leq i \leq C$
  - $J[1] + J[2] + \ldots + J[C] = 2(C-1)$
- *Greedy*:
  - Ambil pelabuhan dengan nilai H minimum dari setiap komponen (agar $J[i] \geq 1$)
  - Sisanya (C-2) pilih pelabuhan-pelabuhan dengan nilai H terkecil dari seluruh sisanya

Subsoal 6 & 7

- Karena Q = 0 hanya perlu pikirkan nilai total yang minimum, tidak perlu konstruksi
- Cari/tentukan semua komponen
  - Untuk Subsoal 6 boleh tidak efisien, $O(N^2)$
  - Untuk Subsoal 7 harus efisien (O(V+E) atau O(N log N)) dengan DFS atau *union-find*
- Terapkan lemma dan hitung total biaya minimum secara *greedy*
  - Jika C = 1 ➔ total = 0
  - Jika 2C - 2 < N ➔ output -1
  - Else:
    - Ambil nilai H terkecil dari dari setiap komponen (C)
    - Ambil C-2 nilai terkecil sisanya
    - Total biaya minimum = total keduanya

Solusi penuh

- Gunakan algoritme pada subsoal 7 untuk menghitung nilai biaya optimal dan menentukan pelabuhan mana saja yang harus dihubungkan
- **Observasi (**asumsi 2C-2 ≤ N**):** Berdasarkan *Pigeonhole Principle*, selalu ada minimal 2 komponen dengan J[i] = 1 pada setiap saat (juga pada saat dilakukan konstruksi seperti di bawah ini)
- Untuk mengkonstruksi jembatan yang diperlukan:
    a. Jika hanya ada dua komponen dan J[1] = J[2] = 1, hubungkan kedua pelabuhannya ➜ selesai
    b. Else: ambil salah satu komponen dimana J[i] = 1 dan hubungkan dengan pelabuhannya dengan salah satu pelabuhan dari komponen dengan J[k] ≥ 2
    c. Gabungkan komponen i dan k, lalu J[k] = J[k] - 1
    d. Ulangi lagi dari langkah **a**.

Membangun Gunungan Buah

Diberikan N kios. Kios i menjual buah berjenis A[i] dengan berat W[i], dan volume V[i]. Anda mempunyai nilai beban b yang awalnya bernilai 1.

Dalam satu waktu, jika Anda berada di kios i, Anda bisa memilih untuk:

- membeli buah di kios i, atau
  Hal ini membutuhkan energi sebesar b.
- pindah ke kios yang bersebelahan.
  Hal ini membutuhkan energi sebesar b*V[i] dan membuat b := b*W[i].

Anda perlu membeli satu buah untuk setiap jenis buah yang ada. Mulai dari buah berjenis 1 sampai jenis buah terbesar. Cari energi minimum yang diperlukan.

Membangun Gunungan Buah (1/6)

- Lakukan coordinate compression terhadap jenis buah.
- Contohnya A = [1, 3, 1, 4, 5, 8, 8] diubah menjadi A = [1, 2, 1, 3, 4, 5, 5].
- Misalkan Mx sebagai jenis buah terbesar setelah melakukan kompresi.

Membangun Gunungan Buah (2/6)

- Gunakan Dynamic Programming bottom-up.
- DP[i] menyatakan energi minimum yang diperlukan apabila Anda membeli buah berjenis A[i] hingga buah berjenis Mx dan memulai rencana Anda dari kios i.
  - Base case
    Jika A[i] = Mx, maka dp[i] = V[i].
  - Transisi
    dp[i] = minimum dari dp[j] * W[i] + |i - j| * W[i] + V[i] untuk setiap j dengan A[j] = A[i] + 1

Membangun Gunungan Buah (3/6)

- Untuk mendapatkan solusi O(N^2) kios-kios dikelompokkan berdasarkan jenis buahnya dan diiterasi dari kios dengan buah berjenis Mx hingga kios dengan buah berjenis 1.
- Untuk mendapatkan solusi O(N) transisi dari solusi sebelumnya cukup dioptimalkan.

Membangun Gunungan Buah (4/6)

- Saat melakukan transisi dengan buah berjenis k, misalkan barisan B sebagai barisan sehingga A[B[i]] = k atau A[B[i]] = k+1 dan B[i] < B[i+1].
- Untuk setiap A[B[i]] = k dan A[B[j]] = k+1, cari

  dp[B[i]] = min(dp[B[j]] * W[B[i]] + |B[i] - B[j]| * W[B[i]] + V[B[i]])

Membangun Gunungan Buah (5/6)

- Saat iterasi i dan j untuk mendapatkan nilai dp[B[i]], misalkan kita hanya perlu memperhatikan kasus ketika B[i] > B[j]. Maka kita hanya perlu mencari

  dp[B[i]] = min(dp[B[j]] * W[B[i]] + (B[i] - B[j]) * W[B[i]] + V[B[i]])

  = min(dp[B[j]] - B[j]) * W[B[i]] + B[i] * W[B[i]] + V[B[i]]
- Perhatikan bahwa, kita hanya hanya perlu memperhatikan min(dp[B[j]] - B[j]) karena variabel lainnya konstan.

Membangun Gunungan Buah (6/6)

- Oleh karena itu, untuk kasus B[i] > B[j] kita cukup iterasi dari dari 1 sampai |B|.
  - Jika A[B[i]] = k+1, bandingkan dp[B[j]] - B[j] dengan nilai minimum sekarang.
  - Jika A[B[i]] = k, assign nilai transisi dengan min(dp[B[j]] - B[j]).
- Hal yang serupa dapat dilakukan untuk kasus B[i] < B[j].

Membangun Jalan

- Ada N toko. Toko i jual kerikil jika T[i] = 1, atau aspal jika T[i] = 2. Satu karung yang dijual di toko i berisi A[i] satuan material.
- Untuk bikin satu jalan, dipilih (L, R), lalu Pak Dengklek beli material dari toko L ke R supaya banyaknya karung yang dibeli di toko-tokonya *strictly increasing*.
- Misal C_1 total kerikil yang dibeli, C_2 total aspal yang dibeli. Panjang ruas jalan tak beraspal adalah $\max(C_1 - C_2, 0)$.
- Pak Dengklek selalu ingin meminimalisasi panjang ruas tak beraspal.
- Hitung jumlah dari minimum panjang ruas jalan untuk setiap $1 \leq L \leq R \leq N$, modulo $10^9 + 7$.

Membangun Jalan (1/9)

- Bikin *array* B sehingga:
  - Jika T[i] = 1, B[i] = A[i].
  - Jika T[i] = 2, B[i] = -A[i].
- Sekarang anggap nilai satu karung itu B[i].
- Sekarang nilai C_1 - C_2 sama dengan jumlah nilai B yang dibeli.

Membangun Jalan (2/9)

- Untuk suatu jalan, misal f[L], f[L + 1], f[L + 2], ..., f[R] adalah banyaknya karung yang dibeli di masing-masing toko.
- Maka $1 \leq f[L] < f[L + 1] < f[L + 2] < \ldots < f[R]$ harus berlaku.
- Lalu panjang ruas jalan tak beraspal adalah:

$$\max(f[L] * B[L] + f[L + 1] * B[L + 1] + \ldots + f[R] * B[R], 0)$$

Membangun Jalan (3/9)

- Definisikan sum(x, y) = B[x] + B[x + 1] + B[x + 2] + ... + B[y].
- Mencari panjang ruas jalan tak beraspal sama seperti mencari nilai-nilai g[L], g[L + 1], g[L + 2], ..., g[R] dengan satu-satunya syarat adalah setiap nilai tersebut harus positif, lalu panjang ruas jalan tak beraspal sama dengan:

$$\max(g[L] * sum(L, R) + g[L + 1] * sum(L + 1, R) + ... + g[R] * sum(R, R), 0)$$

Membangun Jalan (4/9)

- Kita ingin meminimalisasi nilai itu.
- Ada dua kasus:
  - Jika setidaknya ada satu nilai sum yang negatif, kita bisa jadikan nilai g untuk suku itu menjadi sebesar mungkin sehingga panjang ruas tak beraspal menjadi 0.
  - Jika tidak ada nilai sum yang negatif, kita bisa jadikan setiap nilai g menjadi 1 untuk meminimalisasi nilainya.
- Kita bisa menggunakan observasi ini untuk menyelesaikan subsoal 5 ($N \leq 400$) menggunakan *brute force* dan subsoal 6 ($N \leq 3000$) menggunakan *brute force* dengan kompleksitas waktu yang lebih kecil.

Membangun Jalan (5/9)

- Mari kita bikin *prefix sum* P dengan P[i] = B[1] + B[2] + B[3] + ... + B[i].
- Bisa dilihat bahwa sum(x, y) = P[y] - P[x - 1].
- Maka s(x, y) negatif jika dan hanya jika P[x - 1] > P[y].

Membangun Jalan (6/9)

- Untuk suatu nilai R yang tetap, misal kita ingin hitung jumlah panjang jalan tak beraspal untuk setiap L yang ada.
- Pertama kita cari indeks k terkanan sehingga $1 \leq k \leq R$ dan $P[k - 1] > P[R]$.
- Kita bisa lihat bahwa:
  - Untuk setiap L dengan $L > k$, tidak akan ada nilai sum yang negatif.
  - Untuk setiap L dengan $L \leq k$, ada nilai sum yang negatif, maka panjang ruas jalan tak beraspalnya 0.
- Maka kita hanya perlu menghitung untuk setiap L dengan $L > k$.

Membangun Jalan (7/9)

- Untuk suatu L dengan L > k, panjang ruas jalan tak beraspal adalah:

$$(P[R] - P[R - 1]) + (P[R] - P[R - 2]) + \ldots + (P[R] - P[L - 1])$$

$$= P[R] * (R - L + 1) - (P[R - 1] + P[R - 2] + \ldots + P[L - 1])$$

- Maka jumlah panjang ruas jalan tak beraspal untuk suatu R tetap dan untuk setiap L dari k + 1 sampai R sama dengan:

$$P[R] * ((R - k) * (R - k + 1) / 2) - (P[k + 1] * 1 + P[k + 2] * 2 + P[k + 3] * 3 + \ldots + P[R] * (R - k))$$

Membangun Jalan (8/9)

- Nilai tersebut bisa dihitung dengan melakukan prekomputasi di awal-awal untuk menghitung nilai-nilai berikut:
  - $P_2[i] = P[1] + P[2] + P[3] + ... + P[i]$
  - $P_3[i] = P[1] * 1 + P[2] * 2 + P[3] * 3 + ... + P[i] * i$
- Kemudian untuk suatu R dan k, kita bisa mendapatkan jumlahnya dalam kompleksitas waktu O(1) menggunakan nilai-nilai tersebut.

Membangun Jalan (9/9)

- Untuk mencari indeks k terkanan untuk setiap R sehingga $1 \leq k \leq R$ dan $P[k - 1] > P[R]$, kita bisa mengiterasikan *array* P dari kiri ke kanan sambil menyimpan sebuah struktur data *stack* yang menyimpan elemen-elemen pada *array* P yang sejauh ini lebih besar atau sama dengan semua elemen di kanannya jika hanya mengonsiderasi elemen-elemen pada indeks 1 sampai nilai R sekarang.
- Kompleksitas waktu O(N).
- Kompleksitas memori O(N).

Sampai jumpa di IOI 2023